

Counterpoint: Orchestrating Large-Scale Custom Animated Visualizations

Venkatesh Sivaraman*

Frank Elavsky*

Dominik Moritz*

Adam Perer*

Carnegie Mellon University

ABSTRACT

Custom animated visualizations of large, complex datasets are helpful across many domains, but they are hard to develop. Much of the difficulty arises from maintaining visualization state across many animated graphical elements that may change in number over time. We contribute Counterpoint, a framework for state management designed to help implement such visualizations in JavaScript. Using Counterpoint, developers can manipulate large collections of marks with reactive attributes that are easy to render in scalable APIs such as Canvas and WebGL. Counterpoint also helps orchestrate the entry and exit of graphical elements using the concept of a rendering “stage.” Through a performance evaluation, we show that Counterpoint adds minimal overhead over current high-performance rendering techniques while simplifying implementation. We provide two examples of visualizations created using Counterpoint that illustrate its flexibility and compatibility with other visualization toolkits as well as considerations for users with disabilities. Counterpoint is open-source and available at <https://github.com/cmudig/counterpoint>.

Index Terms: Visualization Toolkits, Animation, Web Interfaces, Software System Structures.

1 INTRODUCTION

Web-based interactive data visualizations allow users to consume and explore large quantities of data across diverse computing environments. Toolkits such as D3 [7] and Vega [25, 24] have greatly accelerated the creation of these online data visualizations, helping software developers perform common tasks such as building statistical graphics, laying out network diagrams, and handling interaction paradigms such as brushing-and-linking. While these tools have been sufficient to create many sophisticated and useful visualizations, it remains difficult to venture beyond these tools’ standard functionality in terms of scalability, animation, or accessibility. For example, many tools for machine learning (ML) aim to visualize model embeddings as scatter plots with thousands to millions of points [26, 27], but implementations of such scatter plots using these toolkits tend to lag when interactions and animations are added. Building a smoothly-animating scatter plot from scratch often results in complex code that is harder to modify later and a chart that is less responsive to end-user accessibility requirements, such as interoperability with assistive technologies.

These limitations arise from the trade-offs inherent in current Web graphics APIs, including Scalable Vector Graphics (SVG), HTML5 Canvas, and WebGL. Using native SVG, for instance, developers can take advantage of event handling and accessibility features built into HTML and CSS; however, rapidly modifying SVG elements results in expensive redraw operations and lowered frame-rates. Conversely, Canvas and WebGL can support smooth rendering at large scales, but they do not directly provide affordances for interaction, animation, and accessibility. As a result, the cost of this

flexibility and performance is often a longer development time and more complex code tied to the rendering framework.

We propose to improve the scalability-complexity trade-off in Web graphics APIs by designing rendering-agnostic data structures for visualization state. A central challenge in software development, *state management* concerns how the state of a user interface is stored, read, and manipulated as a system is used. Most modern reactive web frameworks include some form of state management, such as React/Redux or Svelte Stores. However, these systems do not translate well to creating visualizations, which require tracking the state of large time-varying collections of marks with animated properties. Tools for state management designed specifically for large-scale data visualizations could help developers orchestrate changes across different interactions and animations in the rendering environment they prefer.

We contribute Counterpoint, a state management library that makes it easier to create custom, scalable, animated visualizations. Counterpoint provides data types to represent animatable attributes, marks that contain attribute collections, and render groups that manage a potentially time-varying set of marks with independent animations. Developers can use these objects to represent their visualization’s state, then query their values to simplify rendering code. Furthermore, Counterpoint provides utilities on top of its primary types to facilitate interaction tasks such as zooming and hit-testing. Through a performance benchmark, we demonstrate the potential for Counterpoint to accelerate visualizations to Canvas speeds while maintaining the code simplicity of SVG. Then we present two case studies that demonstrate Counterpoint’s ability to easily express animations and work in tandem with other visualization frameworks.

2 RELATED WORK

2.1 Animation in Large-Scale Data Visualization

More than a simple technique to engage viewers, animation is an essential aspect of visualization design that can help communicate a desired message or reveal useful insights [29]. In recent years Web-based animated visualizations have been used to communicate important and complex data to the public, ranging from global health trends [12] to COVID-19 infographics [23] to interactive election forecasts [4, 8]. In these instances, animation effectively conveys change over time, relative magnitude, and uncertainty. While existing visualization toolkits can help developers create and render these graphics, Counterpoint is designed to help manage complexity in their implementation as the number of animations grows.

An important set of use cases for large-scale animated plots is the visualization of specialized datasets for experts. For example, in machine learning, animations in scatter plots of vector embeddings help model developers understand what neural networks have learned and when they might err [27, 22, 26]. Other systems use interactive animations to help students and developers understand how neural networks transform data [30, 32]. In computational biology, researchers use large scatter plots and network diagrams to understand relationships between genes, proteins, and cell types [20, 33, 34]. Our work aims to reduce the technical burden of developing scalable animations in these bespoke visualizations.

*e-mail: {venkats, fje, domoritz, adamperer}@cmu.edu

2.2 Web Frameworks for Visualization

Several tools exist to help developers create data visualizations on the Web, but state management remains an under-addressed challenge. Most prominently, D3 [7] helps developers create SVG-based data visualizations by storing state directly in the Document Object Model (DOM), which tends to work best for non-reactive properties and is difficult to extend to draw loop-based rendering frameworks. General-purpose graphics libraries such as Pixi.js [1], regl [2], and Two.js [3] can help developers attain higher-performance rendering, but they do not assist with state management and reactivity. Meanwhile, higher-level APIs for data visualization such as Vega-Lite [24] and ECharts [19] require little code to create animated charts, but it can be difficult to render custom visual elements or transitions.

Existing tools for state management tend to be focused on communicating state across components of an application, not on orchestrating interaction types within a single view. For example, reactive frameworks such as React, Vue, and Svelte typically use a companion framework for state management (Redux, Vuex, or Svelte stores) to store global state. Specific to visualization, Mosaic [13] provides a “middle-tier” architecture that in essence provides state management for multiple coordinated views. While these approaches work well for non-animated interfaces with a relatively small number of state elements, our work helps manage visualization state that may not only be reactive and animated, but also distributed over a large, time-varying array of graphical elements.

2.3 Animation Accessibility in Visualization

Accessibility and disability-focused design in data visualization is an emerging area in both research and practice [18, 15, 31]. Animation affects both screen reader users (most commonly users who are blind), whose technology limits their ability to perceive live spatial change in elements, and users with motion/vestibular and photo-sensitive disabilities, who may experience adverse effects (such as dizziness, vertigo, or seizures) when viewing animation [28]. However, while accessibility standards and work in HCI acknowledge the impact of animated visualization on these users, it remains difficult to effectively accommodate their needs with existing toolkits [17, 9, 11]. By helping developers maintain state across animations, Counterpoint can help them better address these concerns.

3 COUNTERPOINT LIBRARY

Counterpoint is a TypeScript library that defines useful types for state management in large-scale animated visualizations. We built Counterpoint through an iterative process that took place alongside the development of two major interactive visualization tools [26, 6], as well as several smaller demo projects. While we were able to use frameworks such as D3, regl, and Svelte to quickly create initial prototypes, the need for a robust tool for state management became apparent as the number of interactions and animations grew. For example, clicking a point in the scatter plot in Emblaze [26] could lead to changes in points’ opacity and radius, the appearance of some connecting lines and the disappearance of others, as well as changes to the zoom transform. Animations played a key role in making these visual changes legible to the user, yet they became prohibitively complicated to implement in combination. We therefore developed the following goals for Counterpoint:

G1. *Help developers manage code complexity by balancing declarative and imperative approaches to state management.* Analogous to reactive web frameworks, Counterpoint allows developers to easily specify visualization state through declarative, dynamically-computed attributes while still permitting imperative approaches when needed for simpler or faster code.

G2. *Facilitate efficient draw loop-based rendering across frameworks.* The draw loop is common to most scalable graphics en-

gines, so we designed Counterpoint’s reactivity and animation logic to work efficiently with a ticker that runs at every frame.

G3. *Expose representations of both intermediate and final visualization states.* Animations rendered using a custom draw loop require access to attribute values at every frame, posing a conceptual issue of how to distinguish between the final *specified* state of the visualization and the *momentary* states that arise during animations. We opted to make both types of state available in Counterpoint.

3.1 Marks, Attributes, and Render Groups

The basic unit of a visualization created with Counterpoint is a **Mark**, referring to a graphical element that represents data in some way [21]. Marks can represent any visual element with a set of potentially time-varying characteristics, each represented by a member **Attribute** object (described below). For example, a point mark could be represented using Attributes such as **x**, **y**, **radius**, and **color**. Marks can be associated with a data object using the **represented** property, allowing developers to easily keep track of which marks are associated with which underlying data elements.

An attribute’s value can either be set directly or computed using a callback function at every frame, allowing users the option of using a declarative or imperative approach (**G1**). For example, in the below snippet the resulting mark has two numerical Attributes representing its position and one string Attribute that changes dynamically as the **x**-coordinate changes:

```
let mark = new Mark('mark-id', {
  x: 250, y: 400,
  color: () => `hsl(${mark.attr('x') % 360}, 60%, 60%)`
})
```

The **animate()** or **animateTo()** methods of a mark or render group execute animations between a current attribute value and a new computed or static value, respectively. In addition to configuring the duration, easing function, and delay for any animation, developers can specify a custom interpolator that should be used to map the animation progress to an intermediate value, such as those provided by **d3-interpolate**. Addressing **G3**, attributes expose both momentary and specified states through different accessors, enabling computations that work independently of animations.

To limit data updates and redraws wherever possible (**G2**), most Counterpoint types share an **advance()** method which synchronizes time across the objects and determines whether the element has changed since the last **advance()** call. A redraw will only occur when at least one attribute’s **advance()** method returns **true**. To further improve efficiency, render groups track changes to the marks they contain through internal listener callbacks, so they can efficiently detect which marks need to be advanced at each frame.

Consistent with Counterpoint’s focus on state management alone, none of these objects make assumptions on how they will be rendered; marks and attributes can represent any types of time-varying properties that the developer would like to track.

3.2 Choreographing Mark Entry and Exit

A frequent challenge when implementing interactive visualizations is managing the animated entry and exit of marks. For example, while D3’s **transition** operator can be applied to **enter** and **exit** selections, it can be difficult to specify how elements should behave when they need to re-enter while they are exiting or vice versa. Therefore, Counterpoint includes a system for **staging** marks that explicitly accounts for interruptible entry and exit animations.

For a simple example, imagine we are building a scatter plot showing cars as they were manufactured over time (Fig. 1). While a non-animated version of this visualization could easily be created using off-the-shelf tools like Vega-Lite [24], we may want to make the chart more engaging by animating the alpha and radius when

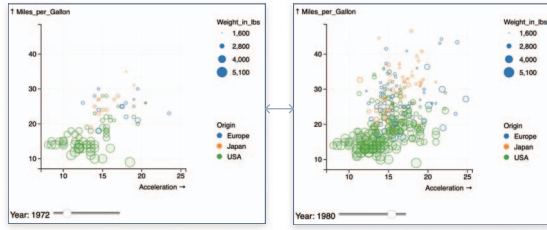


Figure 1: A visualization using the cars dataset [5] which, when animated, demonstrates how staging helps differentiate the specified state (all cars produced before the current year) and the momentary state (marks that are entering and/or exiting based on the previous selection). The animated version is one of the examples at <https://dig.cmu.edu/counterpoint>.

bubbles are added or removed as the user changes the year. This can quickly become complicated to implement in existing frameworks because their APIs do not expose transient states, making it difficult to handle common situations such as moving the slider too quickly or receiving other interactions while animations are still playing.

To implement this chart using Counterpoint, we first create a `MarkRenderGroup` that will contain marks for all cars produced before the currently-selected year. We configure the render group’s staging behavior using up to three function callbacks: `initialize(mark)`, to prepare a new `Mark` for animated entry (e.g. setting its alpha and radius to zero); `enter(mark)`, to animate the mark’s entry (e.g. animating the alpha and radius up) and return a `Promise` that resolves when the animation is complete; and `exit(mark)`, to hide the mark and return a `Promise`. To implement the draw loop, we draw all “onstage” marks by iterating over the stage property of the render group. Addressing G3, staging allows us to specify visualization state on the render group while still being able to access all marks that are actually visible due to the momentary state.

3.3 Tools for Performance and Responsiveness

Once a developer has implemented a visualization using Counterpoint’s data types, they can take advantage of helpers that operate on Counterpoint structures to speed up useful visualization tasks:

Spatial lookup for interaction. While SVG can leverage JavaScript event listeners to respond directly to user interactions on an element, Canvas and WebGL APIs only detect interaction events at the level of the `<canvas>` element, not at individual marks. Counterpoint’s `PositionMap` class uses a spatial hashing approach to index mark locations, enabling constant-time lookup on average and linear-time creation of the hash table when coordinates change.

Zooming and following marks. Counterpoint provides a `Scales` class to facilitate smooth user-initiated and programmatic zoom and pan. These scales use `Attribute` objects to track the current zoom transform, so the scales can be smoothly animated just like marks. Scales can also take advantage of attributes’ reactivity to zoom to, center, or follow specific sets of marks.

Responding to user preferences. Responding to users’ system-wide accessibility settings, such as reduced motion or increased contrast, can help a broader range of people comfortably use visualizations. Counterpoint represents these settings in a `RenderContext` type which can be used like a reactive `Attribute`. Developers can access these attribute values in the draw loop and update the rendering.

Running animations in shaders. While `Attribute.get()` is designed for draw loops that are called on the CPU, additional optimization using WebGL can enable implementations that perform animation directly on the GPU. Attributes can be designated as

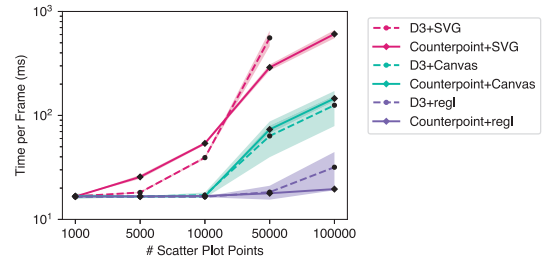


Figure 2: Average frame time for 5-second scatter plot animations using D3 and Counterpoint-based implementations. Shaded intervals represent 1 standard deviation around the mean over 20 trials.

“preloadable,” to specify that when requested, they should return not the current value, but instead four values representing the initial and final values and times for the attribute’s animation. These four values can be sent to the GPU at the start of an animation, so a GPU shader can compute the intermediate values at each frame using only the current time.

4 PERFORMANCE EVALUATION

We conducted a benchmark analysis to measure the performance overhead incurred by Counterpoint’s state maintenance compared to commonly-used approaches. We implemented six versions of the same animated scatter plot, using D3 and Counterpoint with SVG, Canvas, or regl [2] for rendering.¹ In each plot, we measured the duration of each frame of a 5-second animation in which each point moved to a new random location. Additionally during each animation, 25% of the points were removed from the plot using a fade-out transition while an equal number of new points were transitioned in. Evaluations were conducted on a MacBook Pro with an M2 Pro chip (16-core integrated GPU).

As shown in the chart of frame time results in Fig. 2, Counterpoint’s performance is comparable to or better than D3 across rendering frameworks, particularly in larger plots. The D3+SVG implementation scales poorly past 5,000 points, and it fails to load for the largest plot size. With Canvas rendering, the Counterpoint is almost identical to the D3 implementation for 10,000 points or fewer and within 1.2x for larger numbers of points. Note that the D3 implementation uses hardcoded animation logic on a shared timer for all marks, reducing the number of necessary function calls but making the code harder to modify.

While D3 requires taking a substantially different approach when switching from SVG to Canvas or regl, comparison of the implementations² shows how the visualization state logic largely remains the same regardless of renderer. Consistency across different rendering approaches could help developers quickly swap in a higher-performance renderer when deemed necessary, or even support multiple renderers for different browsing environments.

5 USE CASES

5.1 Visualizing Citation Networks Over Time

We demonstrate how Counterpoint enables large-scale visualizations with multiple coordinated animations using an example built on the Visualization Publications Dataset [16], shown in Fig. 3. To allow users to explore how visualization papers have influenced each other over time, we would like to visualize both semantic

¹For D3+Canvas, we adapted the approach described in this tutorial: <https://www.bocoup.com/blog/smoothly-animate-thousands-of-points-with-html5-canvas-and-d3>.

²https://github.com/cmudig/counterpoint/tree/main/examples/performance_eval



Figure 3: Visualizing citation networks and semantic similarity in IEEE VIS papers. Animations of mark opacity, connection lines, and reactive zoom help users trace lineage across papers. The interactive version is available at <https://dig.cmu.edu/counterpoint>.

similarities and citation-based connections between papers. Therefore, we compute a 2-dimensional layout of the 3,620 papers in the dataset using sentence embeddings of their titles. Then, we construct a bubble chart where the color of each node corresponds to its year of publication and the size represents the number of citations, as shown in Fig. 3 (left). As the user changes the year, we use Counterpoint’s stage mechanism to add publications to the plot that were released prior to the selected year.

While this gives us an overview of the dataset over time, interaction is necessary for deeper exploration of the papers. Using Counterpoint’s *PositionMap* we implement hover and select interactions that reveal edges between the current paper and any papers that cite it. We also leverage *Scales* to zoom to the vicinity of the selection, and dynamically adjust the field of view as the year changes to show the paper’s expanding sphere of influence. Finally, to see at a glance how a hovered paper is related to the selected one, we add *Marks* to represent citation paths in a second render group.

5.2 Accessible Visualization of Global Economies

Next, we develop a version of the classic animated Gapminder chart [12] to show Counterpoint’s ability to complement existing frameworks and simplify addressing accessibility considerations. This chart, shown in Fig. 4, consists of standard SVG-based axes and a Canvas-based bubble chart, as well as variable-width lines that animate in when the user hovers on or selects a country. Bubbles and lines are represented as *Marks* whose spatial attributes react to an animatable year attribute.

While rendering this chart in Canvas provides greater control and performance than SVG, it would be difficult to make such a chart responsive and accessible for people with disabilities using existing tools. For users with vestibular or photosensitive disabilities, we configure marks to listen to the *RenderContext*’s *prefersReducedMotion* property, triggering a redraw when the user changes their system-wide motion preference. With reduced motion enabled, we replace line-draw and motion animations with fade transitions (which we achieve by adding and removing copies of each mark from the stage). Furthermore, for screen reader users we implement keyboard and touch-based navigation as well as automatic high- and low-level descriptions using Data Navigator [10].

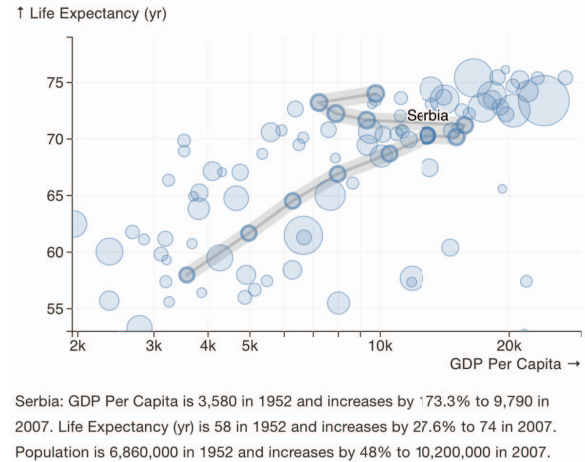


Figure 4: Visualizing global economies from the Gapminder dataset [12]. The user can navigate the chart using multiple modalities and see or hear text descriptions of each mark. The interactive version is one of the examples at <https://dig.cmu.edu/counterpoint>.

We use the marks for each country to generate a navigable structure that selects each country and shows a description of its data values as the user presses arrow keys or swipes through the visualization. We also summarize the country’s overall movement trajectory for a higher level description. Counterpoint helps reduce the amount of code needed to maintain state upon adding these alternative interaction modes.

6 DISCUSSION

We presented Counterpoint, a framework that helps manage complex visualization state and incurs minimal performance overhead while enabling the development of feature-rich systems. Beyond the use cases presented, Counterpoint’s architecture makes it suitable for a wide variety of large-scale animated visualizations, including network visualizations, flow charts, choropleths, beeswarm plots, and others. Abstracting state management can allow developers to focus on incorporating important features such as progressive rendering, support for multiple input modalities, and accessibility for people with visual disabilities. These affordances help bridge the gap between the power of scalable Web graphics APIs and the intricacies of engineering usable visualizations.

Like many state management frameworks, Counterpoint is targeted towards highly complex and custom software systems, so its utility for more standard use cases is limited. While implementing simple animated charts is possible and straightforward with Counterpoint, it provides the most benefit when a plot needs thousands of animating marks and multiple forms of interaction and animation. As a future direction, incorporating Counterpoint into existing visualization authoring tools could help developers easily leverage its functionality regardless of plot complexity.

Future work can build on this initial release by incorporating additional transitions and interpolation functions that support common rendering use cases. For example, Counterpoint could facilitate the creation of parametric and sequenced animations such as the mark type transitions developed in DynaVis [14]. Counterpoint’s API could also be augmented with simple shortcuts for common application needs, such as configuring staging with fade transitions or live-mapping data elements to marks. We are releasing Counterpoint as open-source (version 1.0.4 as of this publication), and we invite the data visualization community to contribute feedback and improvements.

ACKNOWLEDGMENTS

We thank Eli Kranjec, Donghao Ren, Halden Lin, Yannick Assogba, and Fred Hohman for feedback and contributions to the framework design. This work was supported by a National Science Foundation Graduate Research Fellowship (DGE2140739), and by the Carnegie Mellon University Center of Machine Learning and Health.

REFERENCES

- [1] PixiJS | The HTML5 Creation Engine. <https://pixijs.com/>. 2
- [2] regl. <http://regl.party/>. 2, 3
- [3] Two.js. <https://two.js.org/>. 2
- [4] Election Needles: President. *The New York Times*, Nov. 2020. 1
- [5] vega/vega-datasets, Apr. 2024. <https://github.com/vega/vega-datasets>. 3
- [6] A. Boggust, V. Sivaraman, Y. Assogba, D. Ren, D. Moritz, and F. Hohman. Compress and Compare: Interactively evaluating efficiency and behavior across ML model compression experiments, 2024. doi: 10.48550/arXiv.2408.03274 2
- [7] M. Bostock, V. Ogievetsky, and J. Heer. D³ Data-Driven Documents. *IEEE Transactions on Visualization and Computer Graphics*, 17(12):2301–2309, Dec. 2011. doi: 10.1109/TVCG.2011.185 1, 2
- [8] A. Bycoffe and D. Wasserman. What would it take to flip states in the 2016 election? *FiveThirtyEight*, Dec. 2015. <http://projects.fivethirtyeight.com/2016-swing-the-election/>. 1
- [9] F. Elavsky, C. Bennett, and D. Moritz. How accessible is my visualization? evaluating visualization accessibility with chartability. *Computer Graphics Forum*, 41(3):57–70, June 2022. doi: 10.1111/cgf.14522 2
- [10] F. Elavsky, L. Nadolskis, and D. Moritz. Data Navigator: An accessibility-centered data navigation toolkit. *IEEE Transactions on Visualization and Computer Graphics*, 30(1):803–813, Jan. 2024. Place: Los Alamitos, CA, USA Publisher: IEEE Computer Society. doi: 10.1109/TVCG.2023.3327393 4
- [11] D. Fan, A. Fay Siu, H. Rao, G. S.-H. Kim, X. Vazquez, L. Greco, S. O’Modhrain, and S. Follmer. The accessibility of data visualizations on the web for screen reader users: Practices and experiences during COVID-19. *ACM Transactions on Accessible Computing*, 16(1):1–29, Mar. 2023. doi: 10.1145/3557899 2
- [12] Gapminder. Gapminder tools. <https://www.gapminder.org/tools/>. 1, 4
- [13] J. Heer and D. Moritz. Mosaic: An architecture for scalable & interoperable data views. *IEEE Transactions on Visualization and Computer Graphics*, 30(1):1–11, Jan. 2024. doi: 10.1109/TVCG.2023.3327189 2
- [14] J. Heer and G. Robertson. Animated transitions in statistical data graphics. *IEEE Transactions on Visualization and Computer Graphics*, 13(6):1240–1247, Nov. 2007. doi: 10.1109/TVCG.2007.70539 4
- [15] S. Hsueh, B. Vincenzi, A. Murdeshwar, and M. C. Felice. Crippling data visualizations: Crip technoscience as a critical lens for designing digital access. In *Proceedings of the 25th International ACM SIGACCESS Conference on Computers and Accessibility*, ASSETS ’23. ACM, New York, 2023. doi: 10.1145/3597638.3608427 2
- [16] P. Isenberg, F. Heimerl, S. Koch, T. Isenberg, P. Xu, C. Stolper, M. Sedlmair, J. Chen, T. Möller, and J. Stasko. vispubdata.org: A metadata collection about IEEE visualization (VIS) publications. *IEEE Transactions on Visualization and Computer Graphics*, 23(9):2199–2206, Sept. 2017. doi: 10.1109/TVCG.2016.2615308 3
- [17] N. W. Kim, G. Ataguba, S. C. Joyner, C. Zhao, and H. Im. Beyond alternative text and tables: Comparative analysis of visualization tools and accessibility methods. *Computer Graphics Forum*, 42(3):323–335, June 2023. doi: 10.1111/cgf.14833 2
- [18] N. W. Kim, S. C. Joyner, A. Riegelhuth, and Y. Kim. Accessible visualization: Design space, opportunities, and challenges. *Computer Graphics Forum*, 40(3):173–188, June 2021. doi: 10.1111/cgf.14298 2
- [19] D. Li, H. Mei, Y. Shen, S. Su, W. Zhang, J. Wang, M. Zu, and W. Chen. ECharts: A declarative framework for rapid construction of web-based visualization. *Visual Informatics*, 2(2):136–146, June 2018. doi: 10.1016/j.visinf.2018.04.011 2
- [20] S. LYI, Q. Wang, F. Lekschas, and N. Gehlenborg. Gosling: A grammar-based toolkit for scalable and interactive genomics data visualization. *IEEE Transactions on Visualization and Computer Graphics*, 28(1):140–150, Jan. 2022. doi: 10.1109/TVCG.2021.3114876 1
- [21] T. Munzner. *Visualization Analysis and Design*. A.K. Peters/CRC Press, 1st ed., 2014. 2
- [22] S. Robertson, Z. J. Wang, D. Moritz, M. B. Kery, and F. Hohman. Angler: Helping machine translation practitioners prioritize model improvements. In *Proceedings of the 2023 CHI Conference on Human Factors in Computing Systems*, CHI ’23. ACM, New York, 2023. doi: 10.1145/3544548.3580790 1
- [23] H. Samet, Y. Han, J. Kastner, and H. Wei. Using animation to visualize spatio-temporal varying COVID-19 data. In *Proceedings of the 1st ACM SIGSPATIAL International Workshop on Modeling and Understanding the Spread of COVID-19*, pp. 53–62. ACM, Seattle, Nov. 2020. doi: 10.1145/3423459.3430761 1
- [24] A. Satyanarayan, D. Moritz, K. Wongsuphasawat, and J. Heer. Vega-Lite: A grammar of interactive graphics. *IEEE Transactions on Visualization and Computer Graphics*, 23(1):341–350, Jan. 2017. doi: 10.1109/TVCG.2016.2599030 1, 2
- [25] A. Satyanarayan, R. Russell, J. Hoffswell, and J. Heer. Reactive Vega: A streaming dataflow architecture for declarative interactive visualization. *IEEE Transactions on Visualization and Computer Graphics*, 22(1):659–668, 2016. doi: 10.1109/TVCG.2015.2467091 1
- [26] V. Sivaraman, Y. Wu, and A. Perer. Emblaze: Illuminating machine learning representations through interactive comparison of embedding spaces. In *Proceedings of the 27th International Conference on Intelligent User Interfaces*, IUI ’22, p. 418–432. ACM, New York, 2022. doi: 10.1145/3490099.3511137 1, 2
- [27] D. Smilkov, N. Thorat, C. Nicholson, E. Reif, F. B. Viégas, and M. Wattenberg. Embedding Projector: Interactive visualization and interpretation of embeddings. 2016. arXiv: 1611.05469. 1
- [28] L. South and M. A. Borkin. Photosensitive accessibility for interactive data visualizations. *IEEE Transactions on Visualization and Computer Graphics*, 29(1):374–384, 2023. doi: 10.1109/TVCG.2022.3209359 2
- [29] J. Steele and N. Iliinsky. *Beautiful Visualization: Looking at Data through the Eyes of Experts*. O’Reilly Media, Inc., Apr. 2010. 1
- [30] Z. J. Wang, R. Turko, O. Shaikh, H. Park, N. Das, F. Hohman, M. Kahng, and D. H. Polo Chau. CNN Explainer: Learning convolutional neural networks with interactive visualization. *IEEE Transactions on Visualization and Computer Graphics*, 27(2):1396–1406, Feb. 2021. doi: 10.1109/TVCG.2020.3030418 1
- [31] B. L. Wimer, L. South, K. Wu, D. A. Szafrin, M. A. Borkin, and R. Metoyer. Beyond vision impairments: Redefining the scope of accessible data representations. *IEEE Transactions on Visualization and Computer Graphics*, pp. 1–20, 2024. doi: 10.1109/TVCG.2024.3356566 2
- [32] C. Yeh, Y. Chen, A. Wu, C. Chen, F. Viégas, and M. Wattenberg. AttentionViz: A global view of transformer attention, Aug. 2023. doi: 10.48550/arXiv.2305.03210 1
- [33] G. Zhou, O. Soufan, J. Ewald, R. E. W. Hancock, N. Basu, and J. Xia. NetworkAnalyst 3.0: A visual analytics platform for comprehensive gene expression profiling and meta-analysis. *Nucleic Acids Research*, 47(W1):W234–W241, July 2019. doi: 10.1093/nar/gkz240 1
- [34] G. Zhou and J. Xia. OmicsNet: A web-based tool for creation and visual analysis of biological networks in 3D space. *Nucleic Acids Research*, 46(W1):W514–W522, July 2018. doi: 10.1093/nar/gky510 1